

Kmos-0170

Version: 1.0 | Status: Draft

SUMMARY

This specification defines the Plugin & Extension Framework for the Knowledge & Media Operating System (KMOS). The Extension Framework enables organizations to extend KMOS without modifying the platform core. Extensions provide new business functionality while preserving architectural integrity.

PURPOSE

Every Extension SHALL participate in: Capability Discovery; Governance; Event Architecture; Workflow Integration; Knowledge Preservation; Asset Registration; Security; Versioning. Extensions become first-class architectural citizens.

SCOPE

Defines: Extension Architecture, Extension Packages, Registration, Lifecycle, Dependencies, Security, Versioning, Compatibility, Governance, Distribution, Certification, Conformance. Does not prescribe implementation languages or packaging technologies.

ARCHITECTURAL PHILOSOPHY

The KMOS Core remains stable. Organizations extend the platform through Extensions. Extensions SHALL extend the platform and SHALL NOT modify platform behavior by altering core specifications. Platform evolution occurs through composition rather than modification.

EXTENSION PRINCIPLES

Every Extension SHALL be: Self-Contained, Discoverable, Governed, Versioned, Composable, Replaceable, Secure, Observable, Technology Independent, Backward Compatible where practical. Extensions SHALL expose business functionality rather than implementation details.

EXTENSION DEFINITION

An Extension is a governed package that contributes one or more platform capabilities. An Extension MAY contain: Capabilities, Workflow Definitions, Knowledge Models, Vocabulary Packages, Policies, Connectors, Templates, Reference Data, AI Workers, Configuration, Documentation, User Interface Components, Testing Assets, Certification Metadata. An Extension SHALL declare every contributed component.

EXTENSION IDENTITY

Every Extension SHALL possess: Extension Identifier, Canonical Name, Version, Publisher, Owner Organization, Lifecycle State, Certification Level, Publication Date. Identity SHALL remain stable throughout the Extension lifecycle.

EXTENSION MANIFEST

Every Extension SHALL publish a canonical manifest including: Identity, Business Purpose, Provided Components, Dependencies, Required Platform Version, Supported Domains, Supported Capability Versions, Supported Workflow Versions, Configuration Schema, Security Requirements, Licensing Information, Certification Status, Documentation References. The manifest SHALL be machine-readable.

EXTENSION CATEGORIES

Canonical categories include: Capability Extension, Workflow Extension, Knowledge Extension, Connector Extension, Publishing Extension, Media Extension, AI Extension, Governance Extension, Identity Extension, UI Extension, Reference Implementation, Training Package. Future categories SHALL extend this specification.

EXTENSION REGISTRATION

Installation SHALL automatically register the Extension including: Identity, Manifest, Version, Publisher, Capabilities, Workflows, Knowledge Models, Operational Status, Certification, Dependencies. Registration SHALL be idempotent.

DEPENDENCY MANAGEMENT

Extensions SHALL declare dependencies explicitly. Dependency categories include: Platform Version, Capabilities, Workflow Definitions, Knowledge Schemas, Policies, External Services, Other Extensions. Circular dependencies SHALL be prohibited. Dependency validation SHALL occur before activation.

EXTENSION LIFECYCLE

Lifecycle states: Designed → Packaged → Certified → Published → Installed → Registered → Activated → Updated → Deprecated → Archived. Lifecycle transitions SHALL publish canonical events.

CAPABILITY CONTRIBUTIONS

Extensions MAY contribute new Capabilities. Contributed Capabilities SHALL comply with: KMOS-0120, KMOS-0160. All Capability contracts remain governed independently.

WORKFLOW CONTRIBUTIONS

Extensions MAY contribute Workflow Definitions. Workflow contributions SHALL comply with KMOS-0150. Installed workflows SHALL register with the Workflow Registry.

KNOWLEDGE CONTRIBUTIONS

Extensions MAY contribute: Knowledge Types, Ontology Extensions, Vocabulary Packages, Relationship Types, Reference Collections, Educational Models. Knowledge contributions SHALL comply with KMOS-0130.

CONNECTOR CONTRIBUTIONS

Extensions MAY provide Connectors. Examples: Publishing Platforms, Cloud Storage, Identity Providers, Learning Platforms, Enterprise Systems, AI Providers. Connectors SHALL communicate exclusively through canonical events and contracts.

USER EXPERIENCE CONTRIBUTIONS

Extensions MAY contribute presentation components. Examples: Studio Modules, Dashboard Widgets, Editors, Visualization Components, Administration Screens. UI contributions SHALL consume platform services rather than bypass them. Business logic SHALL remain inside Capabilities.

AI CONTRIBUTIONS

Extensions MAY contribute AI workers. AI workers SHALL comply with Capability Standards, Governance Standards, Security Standards, Workflow Standards. Model implementations remain replaceable.

CONFIGURATION

Extension configuration SHALL remain external and support: Business Rules, Operational Parameters, Security Policies, Localization, Feature Flags, Resource Limits. Configuration SHALL be versioned.

SECURITY

Extensions SHALL support platform security including: Publisher Verification, Integrity Verification, Permission Declaration, Secret Management, Audit Logging, Policy Enforcement, Least Privilege. Unsigned Extensions SHALL be explicitly identified.

COMPATIBILITY

Extensions SHALL declare compatibility with: Platform Version, Capability Versions, Workflow Versions, Schema Versions, API Versions. Breaking changes REQUIRE: Major Version, Migration Documentation, Compatibility Analysis.

CERTIFICATION

Extensions SHALL participate in certification evaluating: Architecture Compliance, Business Contracts, Governance, Security, Documentation, Performance, Observability, Testing. Certified Extensions become trusted organizational assets.

DISTRIBUTION

Extensions MAY be distributed through: Private Repositories, Organizational Catalogs, Certified Marketplaces, Local Packages, Offline Archives, Reference Libraries. Distribution mechanisms remain implementation independent.

EXTENSION REGISTRY

Every Extension SHALL be registered within the Extension Registry. Registry records include: Identity, Publisher, Version, Lifecycle, Certification, Dependencies, Capabilities, Knowledge Contributions, Workflow Contributions, Operational State, Documentation. The Extension Registry becomes the authoritative catalog of platform extensions.

CONFORMANCE

An implementation claiming compliance with KMOS-0170 SHALL: Publish a canonical manifest; Register with the Extension Registry; Declare dependencies; Support governance; Support lifecycle management; Support configuration; Support security validation; Support certification; Publish canonical events; Remain compatible with platform contracts. Partial implementations SHALL document unsupported features.

RELATIONSHIP TO PREVIOUS SPECIFICATIONS

KMOS-0100 defined Canonical Objects. KMOS-0110 defined Canonical Events. KMOS-0120 defined Capability Contracts. KMOS-0130 defined Knowledge Objects. KMOS-0140 defined Asset Metadata. KMOS-0150 defined the Workflow Definition Language. KMOS-0160 defined the Capability Development Kit. KMOS-0170 defines how organizations extend the platform while preserving architectural integrity.

ARCHITECTURAL CONSEQUENCES

Because Extensions conform to a common architecture: Organizations customize KMOS without modifying the platform core; Capabilities remain reusable; Knowledge remains interoperable; Applications remain lightweight; Platform upgrades become predictable; Third-party innovation becomes possible; Institutional governance remains consistent across all Extensions. The platform evolves through governed extension rather than fragmentation.

REFERENCE EXTENSION EXAMPLE

Lecture Preservation Suite containing: Speech Recognition Capability, Transcript Correction Capability, Vocabulary Package, Preservation Workflow, Editorial Policies, Archive Connector, Knowledge Templates, Dashboard Components, Documentation, Certification Metadata, Operational Configuration. Installs as a governed unit while each contained component retains its own identity, lifecycle, governance, and version history.

FUTURE EXTENSION SPECIFICATIONS

Examples include: KMOS-1710 Extension Packaging Format; KMOS-1720 Extension Manifest Schema; KMOS-1730 Extension Marketplace Protocol; KMOS-1740 Extension Certification Program; KMOS-1750 Connector Development Standard; KMOS-1760 UI Extension Framework; KMOS-1770 Knowledge Package Format; KMOS-1780 Organizational Extension Catalog. Each SHALL remain compatible with this specification.

NEXT SPECIFICATION

KMOS-0180 defines the API & Integration Standard. If KMOS-0160 defines how Capabilities are engineered, and KMOS-0170 defines how the platform is extended, then KMOS-0180 defines how KMOS communicates with external systems through stable, secure, technology-independent integration contracts. The API & Integration Standard ensures that every external interaction preserves the same principles of canonical objects, canonical events, governance, security, and institutional knowledge that define the KMOS architecture.